

Informe del Trabajo Final

Sistema de automatización del hogar con patrones de diseño

HomeControl

Curso: Diseño de Patrones

Alumno: Berrospi Sosa Damerson

Docente: Atencio Torres Carlos Eduardo

Sección / Ciclo: 41990 5to

Lima - Perú

2026

1. Contexto del Proyecto

1.1. Identificación del Problema

Un hogar inteligente reúne dispositivos de naturaleza distinta (luces, cerraduras, termostatos, sensores) que deben ser controlados desde un punto común. Al construir un sistema así aparecen problemas habituales de desarrollo de software:

- **Creación de objetos dispersa:** si cada parte del programa instancia dispositivos con `new`, el código queda acoplado a clases concretas y cada nuevo tipo obliga a modificar muchos lugares.
- **Un único punto de coordinación:** varias copias del controlador central generarían estados inconsistentes.
- **Equipos legados incompatibles:** una cerradura antigua con interfaz propia (girarLlave()) debe convivir con el resto del sistema sin reescribirla.
- **Reacción a eventos:** un sensor de movimiento debe activar luces u otros elementos sin conocerlos directamente.
- **Complejidad expuesta al cliente:** activar una escena (por ejemplo, «modo noche») no debería exigir que el cliente conozca cada dispositivo.

1.2. Alcance y Objetivos

Funcionalidades implementadas:

- Creación de dispositivos (luz, cerradura y termostato) a partir de un tipo textual mediante una fábrica.
- Registro de dispositivos en un hub central único y ejecución de la acción de todos ellos de forma uniforme.
- Activación de escenas (modoNoche, modoDia) a través de una interfaz simplificada.
- Integración de una cerradura legada mediante un adaptador.
- Notificación automática a una luz cuando un sensor detecta movimiento.
- Representación del comportamiento de un dispositivo según su estado (encendido o apagado).

Objetivos específicos de diseño:

1. Reducir el acoplamiento entre el código cliente y las clases concretas de dispositivos.
2. Poder agregar nuevos tipos de dispositivos y observadores sin modificar el código existente.
3. Centralizar y controlar la creación de objetos y el acceso al hub.
4. Reutilizar componentes legados sin alterarlos.
5. Aplicar patrones GoF de las tres categorías (creacionales, estructurales y de comportamiento) junto con principios SOLID y patrones GRASP.

1.3. Tecnología Utilizada

- **Lenguaje:** Java (orientado a objetos).
- **Entorno de desarrollo:** Apache NetBeans.
- **Gestión del proyecto:** Apache Maven.
- **Modelado:** diagramas de clases UML.

- **Organización en paquetes:** abstracciones, dispositivos, fabrica, automatizacion, adapters, legacy, observer, state y principal.

2. Fundamentos de Diseño (Principios SOLID)

2.1. Aplicación de Principios SOLID

Se identifican tres principios críticos para el diseño de HomeControl.

Principio Abierto/Cerrado (OCP). El hub central trabaja únicamente con la abstracción Dispositivo y recorre su lista llamando a ejecutarAccion(). Por ello, agregar un nuevo dispositivo solo exige crear una subclase de Dispositivo, sin tocar HubCentral ni GestorDeEscenas. Lo mismo ocurre con el Observer: se pueden añadir nuevos observadores sin modificar SensorMovimiento.

```
public void ejecutarTodo(){
    for(Dispositivo d : dispositivos){
        d.ejecutarAccion(); // cerrado a modificación, abierto a nuevos dispositivos
    }
}
```

Principio de Responsabilidad Única (SRP). Cada clase tiene una sola razón para cambiar:

- DispositivoFactory solo crea dispositivos.
- HubCentral solo almacena y ejecuta dispositivos.
- GestorDeEscenas solo define escenas.
- SensorMovimiento solo detecta y notifica.
- CerraduraAdapter solo traduce la interfaz legada.

Principio de Segregación de Interfaces (ISP). En lugar de una interfaz grande, se definieron contratos pequeños según la capacidad: IEncendible, IBloqueable e IMedible. Cada dispositivo implementa solo lo que le corresponde: la luz es IEncendible, la cerradura es IBloqueable y el termostato es IMedible; ninguno queda obligado a implementar métodos que no usa.

De forma complementaria, se cumple el **Principio de Sustitución de Liskov** (cualquier subclase de Dispositivo puede usarse donde se espera un Dispositivo) y la **Inversión de Dependencias**: HubCentral depende de Dispositivo y SensorMovimiento depende de Observador, es decir, de abstracciones y no de clases concretas.

2.2. Diseño de Clases Inicial (Diagrama General)

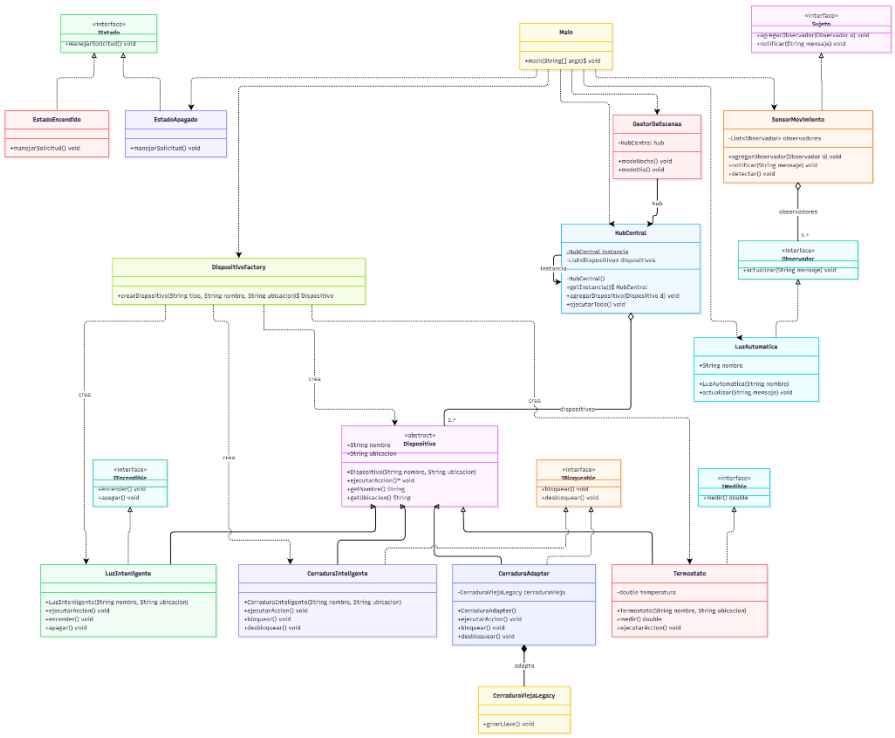


Diagrama 1: Diagrama de clases general del sistema HomeControl

El diseño parte de la clase abstracta Dispositivo, de la que heredan LuzIntenligente, CerraduraInteligente, Termostato y CerraduraAdapter. Las capacidades se expresan mediante las interfaces IEncendible, IBloqueable e IMedible. Alrededor de este núcleo se organizan la fábrica, el hub, el gestor de escenas, el mecanismo de observadores y los estados.

3. Aplicación de Patrones de Diseño (GoF)

3.1. Patrones Creacionales

3.1.1. Singleton: HubCentral

El hub es el punto único de coordinación de la vivienda. Si existieran varias instancias, cada una tendría su propia lista de dispositivos y las escenas se ejecutarían sobre conjuntos distintos. El Singleton garantiza una sola instancia y un acceso global controlado mediante getInstancia(), con constructor privado.

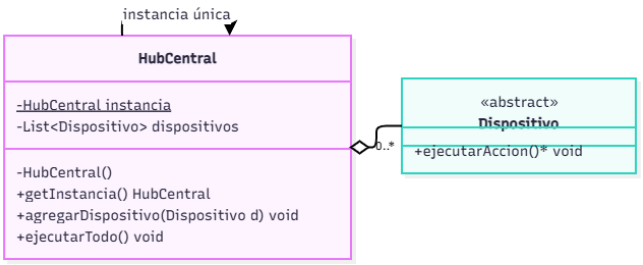


Diagrama 2: Diagrama de clases del patrón Singleton (HubCentral)

```
public class HubCentral {
    private static HubCentral instancia;
    private List<Dispositivo> dispositivos = new ArrayList<>();

    private HubCentral(){}

    public static HubCentral getInstancia(){
        if(instancia == null){
            instancia = new HubCentral();
        }
        return instancia;
    }
}
```

3.1.2. **Fábrica:** DispositivoFactory

La clase centraliza la creación de dispositivos a partir de un tipo textual. El Main solicita un Dispositivo sin conocer las clases concretas, lo que reduce el acoplamiento. Se implementó como una **fábrica simple con método estático**, variante cercana al Factory Method, que concentra en un solo lugar la decisión de qué clase instanciar.

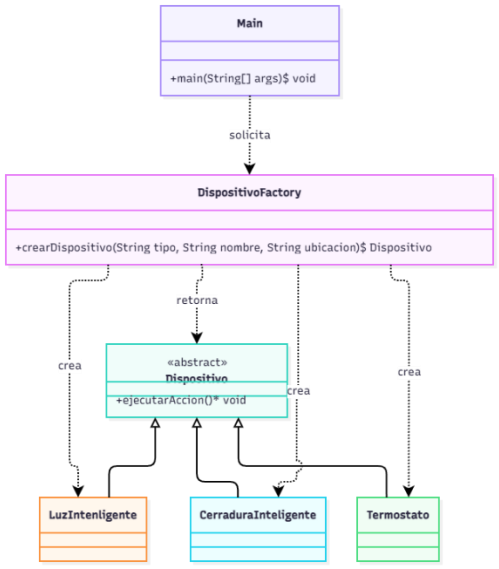


Diagrama 3: Diagrama de clases del patrón Fábrica (DispositivoFactory)

```
public static Dispositivo crearDispositivo(String tipo, String nombre, String
ubicacion){
    switch (tipo.toUpperCase()) {
        case "LUZ":         return new LuzIntenligente(nombre, ubicacion);
        case "CERRADURA":   return new CerraduraInteligente(nombre, ubicacion);
        case "TERMOSTATO": return new Termostato(nombre, ubicacion);
        default: return null;
    }
}
```

3.1.3 Patron Builder: Escena

Las escenas del sistema tienen múltiples configuraciones opcionales (intensidad de luces, temperatura, bloqueo de puertas, alarmas). Usar un constructor con todos estos parámetros sería confuso y propenso a errores. El patrón Builder permite construir escenas paso a paso, con nombres de métodos autoexplicativos (conIntensidadLuces(), conTemperatura()), haciendo el código más legible y flexible.

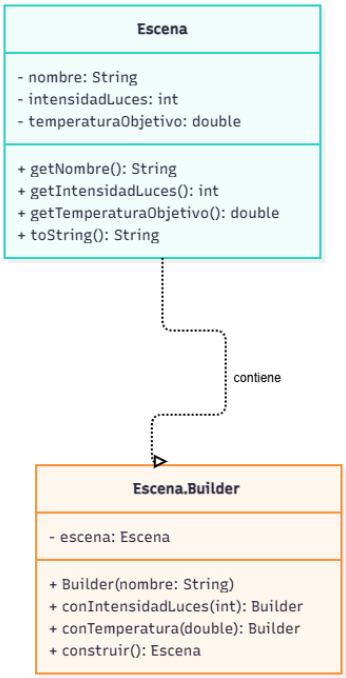


Diagrama 4: Diagrama de clases del patrón Builder (Escena)

```
public static class Builder {
    private Escena escena;

    public Builder(String nombre) {
        escena = new Escena();
        escena.nombre = nombre;
        // Valores por defecto
        escena.intensidadLuces = 100;
```

```
        escena.temperaturaObjetivo = 22.0;
    }

    public Builder conIntensidadLuces(int intensidad) {
        escena.intensidadLuces = intensidad;
        return this; // Permite encadenar métodos
    }

    public Builder conTemperatura(double temperatura) {
        escena.temperaturaObjetivo = temperatura;
        return this;
    }

    public Escena construir() {
        return escena; // Devuelve el objeto final configurado
    }
}
```

3.2. Patrones Estructurales

3.2.1. **Adapter:** CerraduraAdapter

El sistema ya cuenta con una cerradura heredada (CerraduraViejaLegacy) cuyo único método es girarLlave(), incompatible con la interfaz IBloqueable que el sistema espera. El adaptador permite reutilizarla sin modificar su código: hereda de Dispositivo, implementa IBloqueable y contiene internamente la cerradura vieja, traduciendo bloquear() y desbloquear() a llamadas a girarLlave() (adaptador de objeto, por composición).

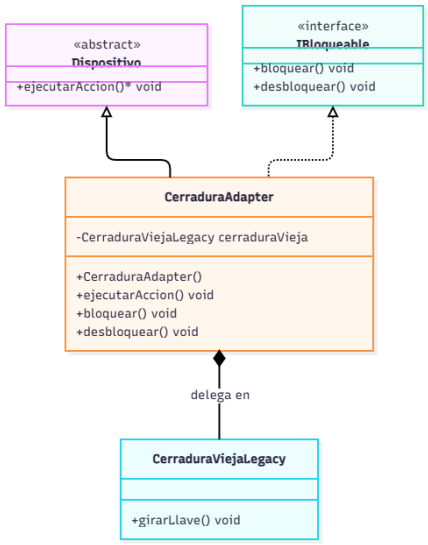


Diagrama 5: Diagrama de clases del patrón Adapter (CerraduraAdapter)

```
public class CerraduraAdapter extends Dispositivo implements IBloqueable {
    private CerraduraViejaLegacy cerraduraVieja = new CerraduraViejaLegacy();

    @Override
```

```
    public void bloquear() {
        cerraduraVieja.girarLlave();
    }

    @Override
    public void desbloquear() {
        cerraduraVieja.girarLlave();
    }
}
```

3.2.2. Facade: GestorDeEscenas

El cliente no necesita conocer el hub ni recorrer los dispositivos para activar una escena. GestorDeEscenas ofrece una interfaz simple (modoNoche(), modoDia()) que oculta la interacción con HubCentral y, a través de él, con todos los dispositivos.

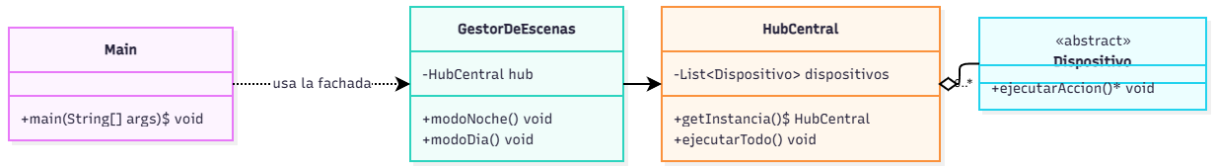


Diagrama 6: Diagrama de clases del patrón Facade (GestorDeEscenas)

```
public class GestorDeEscenas {
    private HubCentral hub = HubCentral.getInstancia();

    public void modoNoche() {
        hub.ejecutarTodo();
    }
}
```

3.3. Patrones de Comportamiento

3.3.1. Observer: SensorMovimiento y LuzAutomatica

Cuando el sensor detecta movimiento, otros elementos deben reaccionar sin que el sensor los conozca. SensorMovimiento mantiene una lista de Observador y los notifica; LuzAutomatica responde en su método actualizar. Esto desacopla emisor y receptores y permite sumar nuevos observadores sin cambiar el sensor.

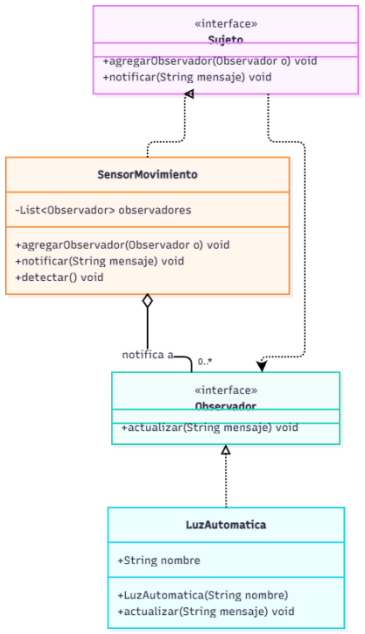


Diagrama 7: Diagrama de clases del patrón Observer

```
public void notificar(String mensaje) {
    for(Observador o : observadores){
        o.actualizar(mensaje);
    }
}

public void detectar(){
    notificar(...);
}
```

3.3.2. State: IEstado, EstadoEncendido y EstadoApagado

El patrón encapsula en clases separadas el comportamiento que corresponde a cada estado de la luz, evitando condicionales del tipo «si está encendida... si está apagada...». IEstado define manejarSolicitud() y cada estado concreto responde de forma distinta ante la misma solicitud.

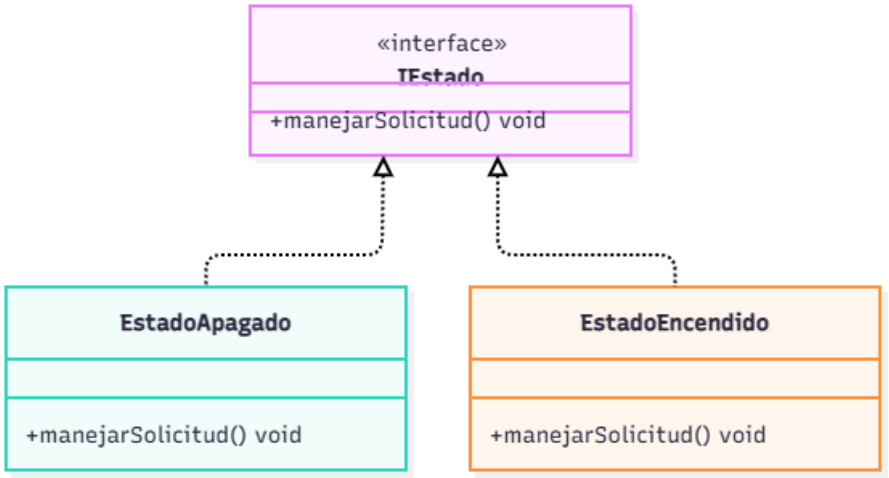


Diagrama 8: Diagrama de clases del patrón State

```
public interface IEstado {
    void manejarSolicitud();
}

public class EstadoApagado implements IEstado {
    public void manejarSolicitud() { /* indica que la luz está apagada y la enciende
*/ }
}
```

4. Asignación de Responsabilidades y Calidad (Patrones GRASP)

4.1. Uso de Patrones GRASP

GestorDeEscenas actúa como controlador de los eventos del sistema: recibe la solicitud de activar una escena y la delega al hub, sin contener lógica propia de los dispositivos. Main actúa solo como cliente que dispara los casos de uso.

Alta Cohesión y Bajo Acoplamiento.

- *Alta cohesión:* cada clase agrupa responsabilidades afines (la fábrica solo crea, el sensor solo detecta y notifica, el hub solo administra dispositivos).
- *Bajo acoplamiento:* HubCentral conoce únicamente la abstracción Dispositivo; SensorMovimiento conoce solo la interfaz Observador; Main obtiene los dispositivos como Dispositivo sin depender de sus clases concretas.

Experto de la información: HubCentral posee la lista de dispositivos, por lo que es quien debe recorrerla y ejecutarla. Del mismo modo, SensorMovimiento conoce a sus observadores y es el responsable de notificarlos.

Creador: DispositivoFactory concentra la creación de los dispositivos, y CerraduraAdapter crea internamente la cerradura legada que utiliza.

Fabricación Pura: DispositivoFactory y GestorDeEscenas no representan conceptos del dominio físico (no son un aparato de la casa), sino clases artificiales creadas para lograr bajo acoplamiento y alta cohesión.

Polimorfismo: Cada dispositivo implementa ejecutarAccion() a su manera (la luz se enciende, la cerradura se bloquea, el termostato mide), y el hub las invoca sin condicionales.

Variaciones Protegidas / Indirección: CerraduraAdapter protege al sistema de la variación de la interfaz legada, interponiendo una clase intermedia.

4.2. Antipatrones Evitados

Antipatrones evitados gracias al diseño:

- **God Class (clase Dios):** las responsabilidades se reparten en clases pequeñas en lugar de una única clase que cree, almacene, ejecute y notifique todo.

- **Spaghetti Code / condicionales dispersos:** el polimorfismo de Dispositivo evita preguntar por el tipo de dispositivo en cada punto del programa; el único switch está acotado a la fábrica.
- **Reinventar la rueda al integrar código legado:** el Adapter reutiliza CerraduraViejaLegacy sin copiarla ni modificarla.
- **Acoplamiento fuerte entre emisor y receptores:** el Observer evita que el sensor dependa de las luces.

5. Conclusiones y Lecciones Aprendidas

5.1. Resultados y Logros

HomeControl resuelve el problema planteado, crea dispositivos de forma centralizada, los gestiona desde un hub único, permite activar escenas con una interfaz simple, integra un equipo legado y reacciona a eventos del sensor. Se aplicaron seis patrones GoF de las tres y se evidenció el uso de principios SOLID y patrones GRASP en la estructura de clases.

5.2. Retos de Diseño

- Decidir qué dispositivos comparten comportamiento y cuáles solo capacidades, resuelto con Dispositivo más interfaces pequeñas.
- Integrar la cerradura antigua sin modificarla, resuelto con el Adapter.
- Mantener al cliente desacoplado de las clases concretas, resuelto con la fábrica y el polimorfismo.
- Coordinar eventos del sensor sin dependencias directas, resuelto con el Observer.

5.3. Consideraciones Futuras

- Incorporar una clase contexto para el patrón State, por ejemplo, que LuzIntenligente mantenga un IEstado y cambie de estado al encender o apagar.
- Diferenciar modoNoche() y modoDia(), por ejemplo, con acciones por escena.
- Reemplazar el switch de la fábrica por un registro de creadores, o por una fábrica abstracta por tipo, para cumplir plenamente OCP.
- Agregar quitarObservador a Sujeto y hacer que LuzAutomatica controle una luz real.
- Usar inicialización segura ante hilos en el Singleton e incluir el hub en GestorDeEscenas para reducir el acoplamiento.
- Añadir pruebas unitarias y corregir detalles de formato en los mensajes de consola.